

SOFTWARE REQUIREMENTS SPECIFICATION

E-Commerce Web Application

Flask (Python) Full-Stack Development Project

Project Type	Client-Based / Industrial Training Project
Backend	Python + Flask
Frontend	HTML, CSS, JavaScript, Bootstrap
Database	MySQL / PostgreSQL
ORM	SQLAlchemy
Authentication	Flask-Login / JWT (for API access)
Document Version	1.0
Date	07 September 2026

Table of Contents

1. Introduction	3
2. User Roles and Access Control	4
3. Functional Requirements	5
4. Database Design	10
5. System Architecture	11
6. Non-Functional Requirements	12
7. Testing Requirements	13
8. Version Control Workflow	14
9. Team Structure and Deliverables	15
10. End-to-End Application Flow	16

1. Introduction

1.1 Purpose

This document defines the functional and non-functional requirements for an E-Commerce Web Application built on Python/Flask. It is intended as the reference specification for the development team, project leads, and the client, describing what the system must do before implementation begins.

1.2 Project Objective

The client requires an online shopping platform where customers can browse products, add them to a cart, place orders, make payments, track deliveries, and request returns. Administrators must be able to manage products, inventory, customers, orders, payments, discounts, and reports. The application is designed to be deployment-ready for real customer use.

1.3 Technology Stack

Layer	Technology
Backend	Python, Flask
Frontend	HTML, CSS, JavaScript, Bootstrap
Database	MySQL / PostgreSQL
ORM	SQLAlchemy (Flask-SQLAlchemy)
Authentication	Flask-Login (web sessions); JWT (API endpoints)
Version Control	Git / GitHub

2. User Roles and Access Control

The system implements Role-Based Access Control (RBAC) with two primary roles. A customer must never be able to reach an admin route, including by direct URL entry.

2.1 Customer Role

- Register and log in
- Browse and search products
- Add products to cart and wishlist
- Place orders and make payments
- Track orders and request returns
- Submit product reviews

2.2 Admin Role

- Manage customers, products, and categories
- Manage inventory and stock levels
- Manage orders and update order status
- Manage coupons and discounts
- Moderate reviews
- Process returns and refunds
- View reports and analytics dashboard

3. Functional Requirements

3.1 Registration

Required fields: First Name, Last Name, Email, Mobile Number, Password, Confirm Password.

Validation rules:

- Email must be valid and unique
- Mobile number must be valid
- Password must meet complexity requirements
- Password and confirmation must match
- Passwords must never be stored as plain text — only a secure hash is stored

3.2 Login

Customer submits email and password. On success the customer is redirected to their dashboard; on failure a generic “Invalid email or password” message is shown. Sessions must be handled securely (e.g. signed session cookies, appropriate expiry).

3.3 Forgot Password

Flow: Forgot Password → Enter Email → Verification → Reset Password → Login. For the initial build, a development SMTP service or a mock reset mechanism is acceptable.

3.4 Customer Profile

Authenticated customers can view/edit their profile, change mobile number and password, manage saved addresses, and view order history, via a “My Account” section (Profile, My Orders, Wishlist, Addresses, Reviews, Change Password, Logout).

3.5 Product Catalog

Each listing displays: product image, name, price, discount, final price, rating, and stock availability.

3.6 Product Details

The product detail page shows images, brand, description, price, discount, final price, rating and reviews, available stock, specifications, warranty, and available variants, with an Add to Cart action.

3.7 Product Search

Search should match against product name, brand, category, and relevant keywords, returning ranked results.

3.8 Product Filtering

Customers can narrow results by category, price range, brand, and rating (e.g. “4★ & above”).

3.9 Product Sorting

Supported sort orders: Price (Low → High), Price (High → Low), Newest, Popularity, Rating.

3.10 Categories and Subcategories

Products are organized into a category/subcategory hierarchy (e.g. Electronics → Mobiles, Laptops, Headphones, Cameras). Admin can create and manage categories.

3.11 Product Variants

Products may have variant dimensions (e.g. Color, Size). Stock must be tracked per variant combination, not as a single product-level count (e.g. Black+M = 10 units, White+M = 0 units).

3.12 Shopping Cart

Customers can add items to the cart, increase/decrease quantity, remove items, continue shopping, and proceed to checkout. Subtotal is calculated from unit price $\times$ quantity across all cart items.

3.13 Stock Validation

If a customer requests a quantity greater than available stock, the system must reject the request and display the actual available quantity. This validation must be enforced on the backend, not only in the browser.

3.14 Wishlist

Customers can add and remove products from a wishlist, and move wishlist items into the cart.

3.15 Address Management

Customers can save multiple addresses (e.g. Home, Office, Other), each with Name, Mobile, Address Line, City, State, Postal Code, and Country, and select one during checkout.

3.16 Checkout

Checkout sequence: Address $\rightarrow$ Cart Items $\rightarrow$ Coupon $\rightarrow$ Delivery $\rightarrow$ Payment $\rightarrow$ Order Confirmation. The customer must be able to review the full order before placing it.

3.17 Coupon System

Admin can create coupons with a code, discount percentage, minimum order value, maximum discount cap, and expiry date. On redemption the backend validates that the coupon exists, is active, is not expired, the minimum order value is met, and any usage limits are not exceeded.

3.18 Order Creation

On successful checkout, an order record is created with a unique Order ID, date, total amount, and initial status (e.g. Processing).

3.19 Payment Module

For the initial build, support Cash on Delivery and a mock payment gateway; a real gateway is integrated only if the client requires it. Each payment record should store Payment ID, Order ID, Amount, Payment Method, Transaction ID, Payment Status, and Payment Date.

Payment statuses:

- Pending
- Success
- Failed
- Refunded

3.20 Order Status Workflow

Stage	Description
Placed	Order submitted by customer
Confirmed	Order accepted for processing
Processing	Order being prepared
Shipped	Order dispatched
Out for Delivery	Order with delivery partner
Delivered	Order received by customer
Cancelled / Return Requested / Returned / Refunded	Exception flows

3.21 Order Tracking

Customers can view a step-by-step status timeline for each order (e.g. Placed ✓, Confirmed ✓, Processing ✓, Shipped ✓, Out for Delivery ○, Delivered ○).

3.22 Order Cancellation

Order Status	Cancellation Allowed?
Placed	Yes
Processing	Conditional, per business rule
Shipped	No
Delivered	No

3.23 Returns

Flow: Order Delivered → Request Return → Select Reason → Submit → Admin Reviews → Approved / Rejected.

Accepted return reasons:

- Damaged product
- Wrong product
- Product not as expected
- Defective product

3.24 Refund

Flow: Return Approved → Product Returned → Refund Initiated → Refund Completed. The refund status must be tracked at every stage.

3.25 Product Reviews

Only customers who purchased a product may normally review it. Admin can moderate and remove inappropriate reviews.

3.26 Inventory Management

Admin can view current stock per product/variant, and the system should flag low-stock items automatically (e.g. below a configurable threshold).

3.27 Admin – Product Management

- Add / edit / deactivate products
- Upload product images
- Set price and discount
- Manage stock and variants

3.28 Admin Dashboard

Displays summary metrics — total customers, products, orders, revenue, pending orders, low-stock count, and open returns — with supporting charts.

3.29 Customer Management (Admin)

Admin can view, search, and activate/deactivate customer accounts and view a customer's order history, without exposing unnecessary sensitive information.

3.30 Order Management (Admin)

Admin can view Order ID, Customer, Order Date, Amount, Payment Status, and Order Status for every order, and update order status.

4. Database Design

Core entities and their relationships:

Entity	Purpose
users	Customer and admin accounts
categories	Product category hierarchy
products	Product catalog
product_images	Images per product
product_variants	Variant combinations (e.g. color/size) and their stock
inventory	Stock levels
cart / cart_items	Active shopping cart contents
wishlist	Saved-for-later products
addresses	Customer delivery addresses
orders / order_items	Placed orders and their line items
payments	Payment transactions and status
coupons	Discount codes and rules
reviews	Product ratings and reviews
returns / refunds	Post-delivery return and refund records

Core relationship chain: User → Orders → Order Items → Products.

Employees implementing the schema should be explicit about one-to-one, one-to-many, and many-to-many relationships, foreign keys, joins, and constraints, using Flask-SQLAlchemy rather than hand-written SQL wherever practical.

5. System Architecture

5.1 Application Structure

The project must not be implemented as a single app.py file. A modular layout is required:

- ecommerce/app/__init__.py — application factory
- ecommerce/app/models/ — SQLAlchemy models
- ecommerce/app/routes/ — Blueprints
- ecommerce/app/services/ — business logic
- ecommerce/app/templates/ , ecommerce/app/static/ — frontend assets
- ecommerce/app/utils/ — helpers
- ecommerce/migrations/ — database migrations
- ecommerce/tests/ — automated tests
- ecommerce/config.py, ecommerce/requirements.txt, ecommerce/run.py

5.2 Flask Blueprints

Functionality is separated into independent Blueprints: auth, products, cart, orders, admin, payments, and reviews — to keep the codebase modular as it grows.

5.3 API Endpoints (indicative)

Method	Endpoint	Purpose
GET	/api/products	List products
GET	/api/products/<id>	Product detail
POST	/api/cart	Add item to cart
PUT	/api/cart/<id>	Update cart item
DELETE	/api/cart/<id>	Remove cart item
POST	/api/orders	Create order
GET	/api/orders	List orders

API documentation covering request/response formats and authentication requirements must be delivered alongside the code.

6. Non-Functional Requirements

6.1 Security

- Passwords are hashed, never stored in plain text
- All database access uses parameterized queries / the ORM — no unsafe string-built SQL
- Forms are protected against CSRF
- Admin routes (e.g. /admin/products) are protected by authorization checks, not just hidden links
- Uploaded product images are validated; arbitrary file types are rejected

6.2 Validation

All validation (quantity, price, coupon rules, stock limits, etc.) must be enforced server-side; client-side validation is a convenience only, never the sole safeguard.

6.3 Error Handling

Code	Meaning	Customer-Facing Behaviour
404	Product not found	Friendly “not found” page
401	Unauthorized	Redirect to login
403	Access denied	Friendly permission-denied page
500	Internal server error	Generic error page — no stack trace shown

6.4 Pagination

Product listings must be paginated (e.g. 20 items per page with page navigation) rather than loading the full catalog at once.

6.5 Logging

Key events (login, order creation, payment result, etc.) are logged with timestamps. Sensitive data such as passwords must never be written to logs.

7. Testing Requirements

Functionality must be verified with documented test cases rather than informal confirmation. Indicative cases:

Test Case	Expected Result
Valid login	Login successful
Invalid password	Error displayed
Add product to cart	Product added
Quantity requested > available stock	Request rejected
Valid coupon applied	Discount applied
Expired coupon applied	Coupon rejected
Successful payment	Order created
Failed payment	Order/payment status handled correctly

Deliverables include the test case list, a test execution report, and a bug report with final results.

8. Version Control Workflow

Git usage is mandatory. Branching model: main → develop → feature/<module> (e.g. feature/product-module, feature/cart-module, feature/order-module).

Standard workflow per feature:

- Create feature branch
- Develop the feature
- Commit changes
- Push to remote
- Open a Pull Request
- Code review
- Merge to develop

9. Team Structure and Deliverables

9.1 Suggested Team Structure

Team	Module
Team 1	Authentication & Users
Team 2	Products & Categories
Team 3	Cart & Wishlist
Team 4	Checkout & Orders
Team 5	Payment
Team 6	Admin Dashboard
Team 7	Reviews & Returns
Team 8	Testing & QA

One Team Lead should coordinate integration across modules.

9.2 Final Deliverables

Documentation:

- SRS / Requirement Document (this document)
- Use Case Diagram
- ER Diagram
- Database Design
- API Documentation
- Installation Guide
- User Guide

Development:

- Flask source code
- Database
- Frontend
- APIs
- Authentication
- Admin panel

Testing:

- Test cases
- Test execution report
- Bug report
- Final test results

Version Control:

- Git repository
- Branches

- Commits
- Pull requests

10. End-to-End Application Flow

10.1 Customer Journey

Registration → Login → Browse Products (via Search / Filter) → Product Details → Wishlist or Add to Cart → Checkout (Address + Coupon) → Payment → Order Created → Order Tracking → Delivered → Review and/or Return → Refund (if applicable).

10.2 Admin Scope

Dashboard, Customers, Products, Categories, Inventory, Orders, Payments, Coupons, Reviews, Returns, and Reports.

This project gives the development team practical exposure to Flask, Python, relational database design, authentication, CRUD operations, entity relationships, REST APIs, frontend integration, business-rule enforcement, security practices, structured testing, and a professional Git workflow — not just basic CRUD.